

ABSTRACT

This mini project presents a software-based real-time Intrusion Detection System (IDS) titled ****Intrusion Activity Detection & Attack Trace System****. The primary aim of the project is to monitor host log activity continuously, detect suspicious behavior using rule-based and dataset-informed logic, correlate related events into meaningful attack chains, and present the results through a live web dashboard and downloadable incident report.

The system is designed for both demonstration and practical learning in cyber security. It supports real-time and historical monitoring modes. On Windows systems, it reads event activity from Event Viewer logs. On Linux systems, it can monitor system and web server logs such as authentication logs, syslog, and access logs. In addition to live host monitoring, the system also includes bundled sample logs and safe demo scripts so that the IDS can be tested even when real suspicious activity is not available.

*The detection engine is based on two major approaches. The first is ****OWASP Top 10 inspired rule-based detection****, where the system identifies patterns associated with Broken Access Control, Cryptographic Failures, Injection attacks, and Authentication Failures such as brute-force attempts. The second approach is ****dataset-enhanced detection****, where suspicious IP addresses, known attack signatures, and abnormal request frequency patterns are extracted from CICIDS-style sample data and used to strengthen the detection process.*

TABLE OF CONTENTS

Abstract

List of Tables

List of Figures

List of Symbols, Abbreviations and Nomenclature

Chapter 1: Introduction

1.1 Overview and Motivation

1.2 Objective

1.3 Summary of Similar Application

1.4 Organization of the Project Report

Chapter 2: Software Requirement Analysis

2.1 Requirement Analysis

2.2 Feasibility Analysis

2.3 Modules Description

2.4 Functionalities of Each Module

2.5 Use Case Scenarios

Chapter 3: Software Design

3.1 Data Flow Diagram

3.2 UML Diagrams

3.3 Database Design

Chapter 4: Implementation and User Interface

4.1 Backend Implementation

4.2 Detection Implementation

4.3 Realtime Monitoring Implementation

4.4 User Interface

4.5 Result Analysis

Chapter 5: Software Testing

5.1 Black Box Testing

LIST OF TABLES

Table 2.1 Functional Requirements

Table 2.2 Non-Functional Requirements

Table 2.3 Feasibility Analysis Summary

Table 2.4 Module Description Summary

Table 3.1 Logical Data Storage Design

Table 5.1 Black Box Test Cases

Table 5.2 White Box Test Cases

LIST OF FIGURES

Figure 3.1 Data Flow Diagram Level 0

Figure 3.2 Data Flow Diagram Level 1

Figure 3.3 Use Case Diagram

Figure 3.4 Class Diagram

Figure 3.5 Object Diagram

Figure 3.6 Sequence Diagram

Figure 3.7 Collaboration Diagram

Figure 3.8 E-R Diagram / Logical Storage Diagram

Figure 4.1 Dashboard Home Screen

Figure 4.2 Live Monitoring View

Figure 4.3 Alerts Panel

Figure 4.4 Timeline View

Figure 4.5 Attack Chains View

Figure 4.6 PDF Report Output

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

- IDS: Intrusion Detection System
- API: Application Programming Interface
- UI: User Interface
- OWASP: Open Worldwide Application Security Project
- SQLi: SQL Injection
- DFD: Data Flow Diagram
- UML: Unified Modeling Language
- ER: Entity Relationship
- PDF: Portable Document Format
- WS: WebSocket
- IP: Internet Protocol
- CICIDS: Canadian Institute for Cybersecurity Intrusion Detection System Dataset
- SRS: Software Requirement Specification

CHAPTER 1: INTRODUCTION

1.1 Overview and Motivation

Cyber attacks are becoming more frequent and more sophisticated, while modern systems continuously generate large volumes of logs. Important evidence of attacks is often hidden inside authentication logs, web access logs, operating system event logs, and system alerts. Manual log analysis is time-consuming and error-prone, especially when multiple suspicious events happen close to each other.

The motivation behind this project is to develop a software-based IDS that can automatically monitor host logs, identify suspicious behavior, and present security-relevant findings in a structured and understandable way. Instead of depending only on raw log files, the system transforms events into alerts, timelines, and correlated attack chains. This makes it easier for students, analysts, and evaluators to understand not only that an attack may have happened, but also how the attack progressed.

Another important motivation is educational value. Many IDS demonstrations stop at detection only, but practical incident understanding also requires correlation, visualization, and reporting. This project therefore combines log ingestion, real-time monitoring, rule-based detection, dataset-informed logic, dashboard visualization, and PDF reporting in one complete workflow.

1.2 Objective

The main objectives of this project are:

1. To monitor host and application logs in real time.
2. To detect suspicious activity using OWASP Top 10 inspired rules.
3. To enhance detection using dataset-derived patterns such as suspicious IPs and known signatures.
4. To correlate related alerts into attack chains.
5. To generate a chronological event timeline for easier investigation.
6. To provide a live web dashboard for user interaction and monitoring.
7. To generate a final PDF report summarizing the observed security activity.

1.3 Summary of Similar Application

Several security monitoring and SIEM platforms already exist in industry, such as Splunk, Wazuh, OSSEC, and Elastic Security. These tools provide log ingestion, search, detection, visualization, and alerting. However, such platforms are often enterprise-oriented, complex to configure, or resource-heavy for an academic mini-project.oriented,

complex to configure, or resource-heavy for an academic mini-project. This project is comparatively lightweight and focused on demonstrating the main concepts of intrusion detection in a compact and understandable way. Unlike full enterprise SIEM tools, this system avoids heavy infrastructure dependencies. Instead, it uses FastAPI, rule-based detection, sample dataset pattern extraction, and a custom dashboard. The project is therefore simpler to deploy, easier to explain in an academic setting, and still capable of demonstrating key IDS concepts such as monitoring, detection, event correlation, and reporting.

1.4 Organization of the Project Report

This report is organized into six chapters. Chapter 1 introduces the project, motivation, objectives, and related application summary. Chapter 2 presents the software requirement analysis, feasibility analysis, module descriptions, and use case scenarios. Chapter 3 describes the software design using data flow diagrams, UML diagrams, and logical storage design. Chapter 4 explains implementation details, user interface behavior, and result analysis. Chapter 5 presents black box and white box testing. Chapter 6 concludes the project and suggests possible future improvements.

CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS

2.1 Requirement Analysis

The system requirements were divided into functional and non-functional requirements.

Functional Requirements

Requirement ID	Requirement Description
FR1	The system shall monitor logs from supported sources.
FR2	The system shall parse raw log data into structured events.
FR3	The system shall detect OWASP-related suspicious patterns.
FR4	The system shall detect suspicious activity using dataset-informed signatures and IPs.
FR5	The system shall create alerts with severity and metadata.
FR6	The system shall correlate related alerts into attack chains.
FR7	The system shall display logs, alerts, timeline, and attack chains on a live dashboard.
FR8	The system shall support live monitoring and historical scan modes.
FR9	The system shall generate a PDF report after monitoring stops.
FR10	The system shall provide demo scripts for safe testing.

Non-Functional Requirements

Requirement ID	Requirement Description
NFR1	The system should be modular and easy to maintain.
NFR2	The system should respond quickly to new log events.
NFR3	The dashboard should be simple and easy to understand.
NFR4	The code should be beginner-friendly and readable.
NFR5	The system should run on both Windows and Linux where possible.

2.2 Feasibility Analysis

Technical FeasibilityThe project is technically feasible because the required software stack is lightweight and open source. FastAPI supports REST APIs and WebSockets, Python supports regex-based log parsing and rule evaluation, and ReportLab supports PDF creation. Windows and Linux logs can both be handled using platform-aware logic.

Economic Feasibility

The project is economically feasible because no paid tools or licensed frameworks are required. The main components are Python, FastAPI, and standard open-source libraries.

Operational Feasibility

The system is operationally feasible because it can be executed on a standard laptop and demonstrated safely using built-in sample logs and demo scripts without requiring an enterprise infrastructure.

Schedule Feasibility

The project is schedule-feasible because the modules were developed incrementally: parser, detectors, correlation, timeline, backend APIs, dashboard, and reporting.

2.3 Modules Description

The application is divided into the following modules:

1. ****Log Parsing Module**** Reads raw log input and converts it into structured events.
2. ****OWASP Detection Module**** Applies rule-based detection for selected OWASP Top 10 attack categories.
3. ****Dataset Pattern Module**** Adds suspicious IPs, known signatures, and abnormal request frequency rules from dataset samples.
4. ****Realtime Monitoring Module**** Reads new log events continuously from host or demo sources.
5. ****Correlation Module**** Connects related alerts into attack chains.
6. ****Timeline Module**** Builds a chronological sequence of events.
7. ****API and WebSocket Module**** Exposes logs, alerts, timeline, and report download through APIs and pushes live updates to the dashboard.
8. ****Reporting Module**** Generates the PDF report at the end of monitoring.

2.4 Functionalities of Each Module

- The parser identifies log type and extracts fields such as IP address, path, message, status code, and timestamp.
 - The OWASP detector identifies access control abuse, cryptographic failure indicators, SQL injection patterns, and suspicious login behavior.
 - The dataset module checks suspicious IPs, known signatures, and request bursts.
 - The monitor supports auto mode, live mode, historical scan mode, and demo mode.
 - The correlation engine creates attack chains from related alerts.
 - The timeline builder records a sequence of logs, alerts, and chains.
 - The dashboard shows updated data in near real time using WebSockets and API refresh.
 - The report module produces a final PDF for submission and incident review.

2.5 Use Case Scenarios

Use Case 1: Live Monitoring

- User opens the dashboard.
- User selects Live Monitoring.

- User clicks Start Monitoring.
- System begins monitoring host logs or demo logs.
- Logs are parsed and analyzed.
- Alerts and timeline updates appear in real time.

Use Case 2: Historical Scan

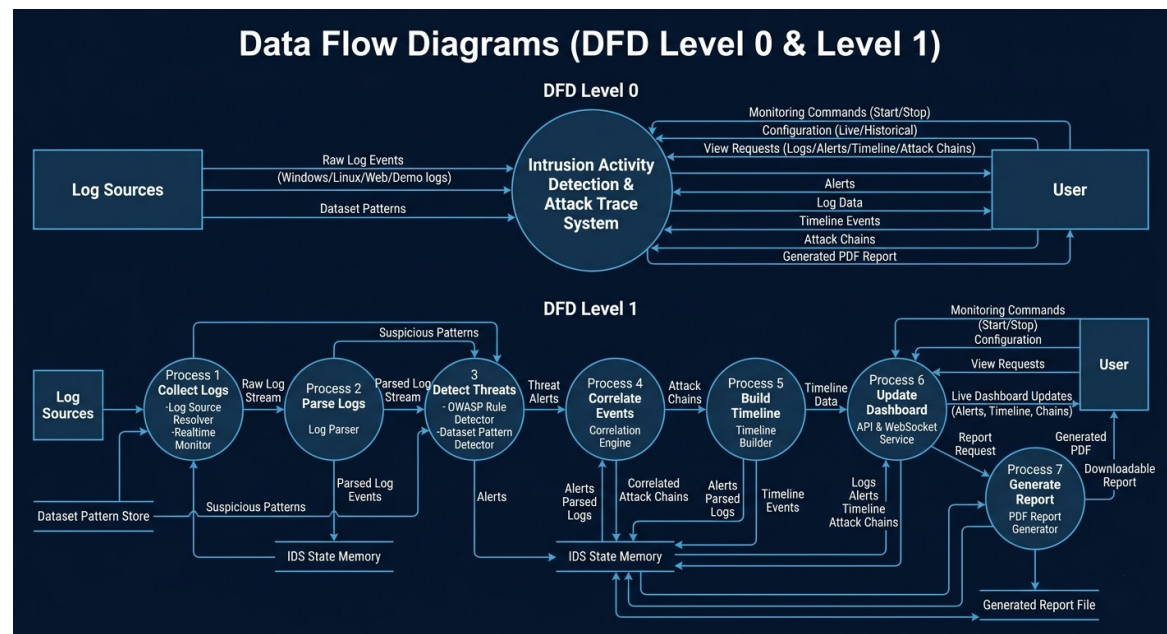
- User selects Historical Scan.
- User clicks Start Monitoring.
- System reads previously stored logs in addition to new incoming events.
- Alerts and attack chains are generated from past evidence.

Use Case 3: Report Generation

- User clicks Stop Monitoring.
- System generates final PDF report.
- User downloads the report from the dashboard.

CHAPTER 3: SOFTWARE DESIGN

3.1 Data Flow Diagram

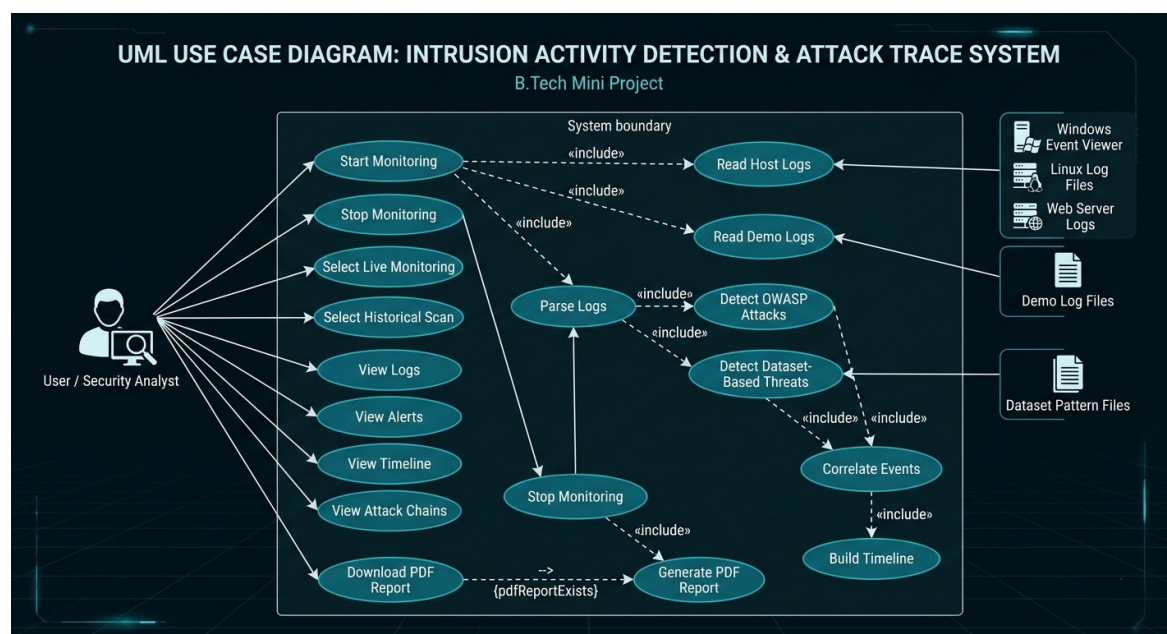


At Level 0, the system accepts log input from host logs or demo logs, processes them through the IDS engine, and outputs alerts, timelines, attack chains, and reports to the user interface.

At Level 1, the IDS engine is expanded into parsing, detection, correlation, timeline generation, API/WebSocket service, and report generation.

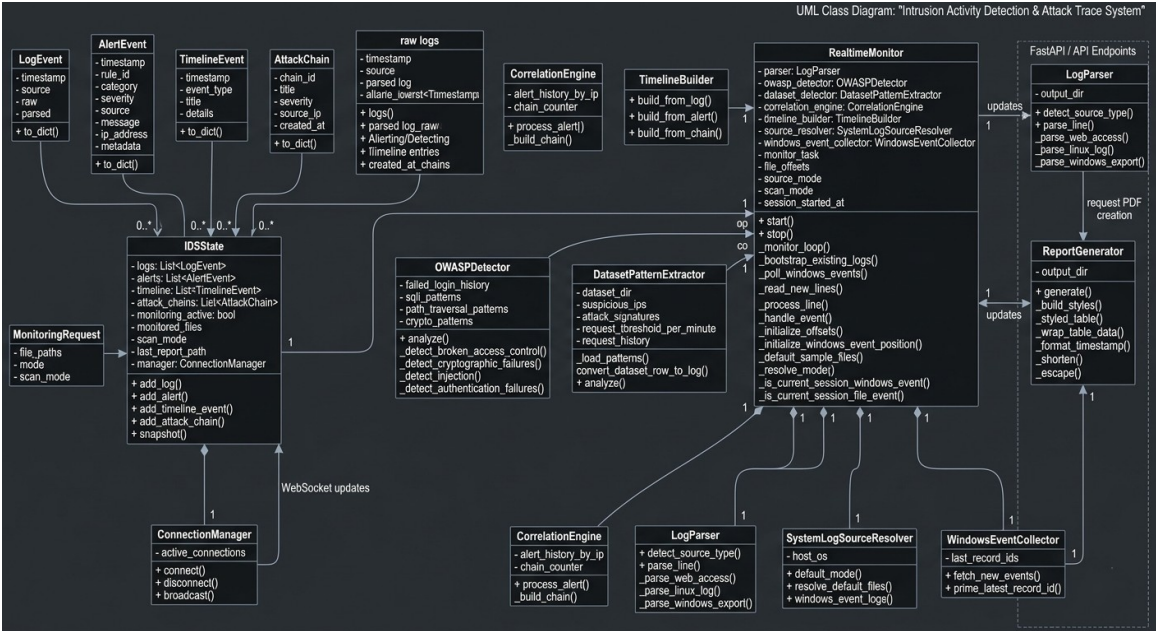
3.2 UML Diagrams

Use Case Diagram



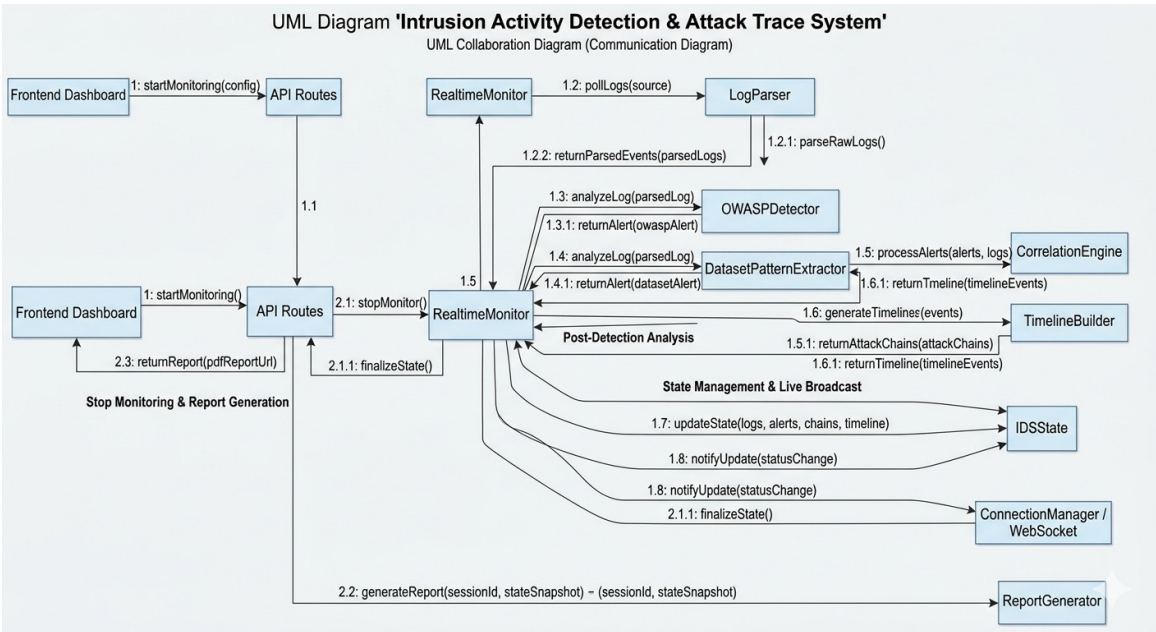
The main actor is the user. The user can start monitoring, stop monitoring, view alerts, inspect timeline, inspect attack chains, run historical scan, and download the report.

Class Diagram



The main classes include 'LogEvent', 'AlertEvent', 'TimelineEvent', 'AttackChain', parser classes, detector classes, state manager, monitor controller, and report generator.

Collaboration Diagram



3.3 Database Design

The implemented prototype does not use an external relational database. Instead, it uses in-memory state for logs, alerts, timelines, and attack chains. This keeps the prototype lightweight and simpler for a mini-project. However, for future scalability, the following logical storage design is proposed.

Proposed Logical Storage Entities

Entity	Main Fields
Logs	log_id, timestamp, source, raw_data, parsed_fields
Alerts	alert_id, timestamp, rule_id, category, severity, message, ip_address
Timeline	timeline_id, timestamp, event_type, title, details
AttackChains	chain_id, severity, source_ip, steps, created_at

No stored procedures are used in the current implementation because the system does not use a persistent SQL database. This is a conscious design choice to keep the project focused on real-time IDS behavior.

CHAPTER 4: IMPLEMENTATION AND USER INTERFACE

4.1 Backend Implementation

The backend is implemented using FastAPI. The main backend entry file initializes the application, configures CORS, serves the frontend, and exposes REST and WebSocket endpoints. Shared runtime state stores logs, alerts, timeline entries, attack chains, selected source mode, and scan mode.

The backend includes:

- parser module for Linux, web, and Windows-style log parsing
- rule-based OWASP detector
- dataset-informed detector
- realtime monitor supporting live and historical scan
- correlation engine
- timeline builder
- PDF report generator

4.2 Detection Implementation

The detection engine is implemented in a modular form.

OWASP Detection

The following categories are implemented:

- A01: Broken Access Control
- A02: Cryptographic Failures
- A03: Injection
- A07: Identification and Authentication Failures

Examples of detection logic:

- `403` or path traversal patterns indicate suspicious access control behavior.
- `TLSv1.0`, weak cipher, or insecure protocol indicators imply cryptographic issues.
- `UNION SELECT` and similar payloads indicate possible SQL injection attempts.
- multiple failed logins followed by a successful login indicate brute-force behavior.

Dataset-Enhanced Detection

The dataset module loads suspicious IPs and signatures from bundled JSON and CSV sample files. It also tracks request frequency to detect bursts of abnormal activity.

4.3 Realtime Monitoring Implementation

Realtime behavior is implemented using an asynchronous monitoring loop. Depending on the operating system and selected mode:

- Windows mode reads Event Viewer logs
- Linux mode tails standard system/web logs
- demo mode uses sample bundled logs

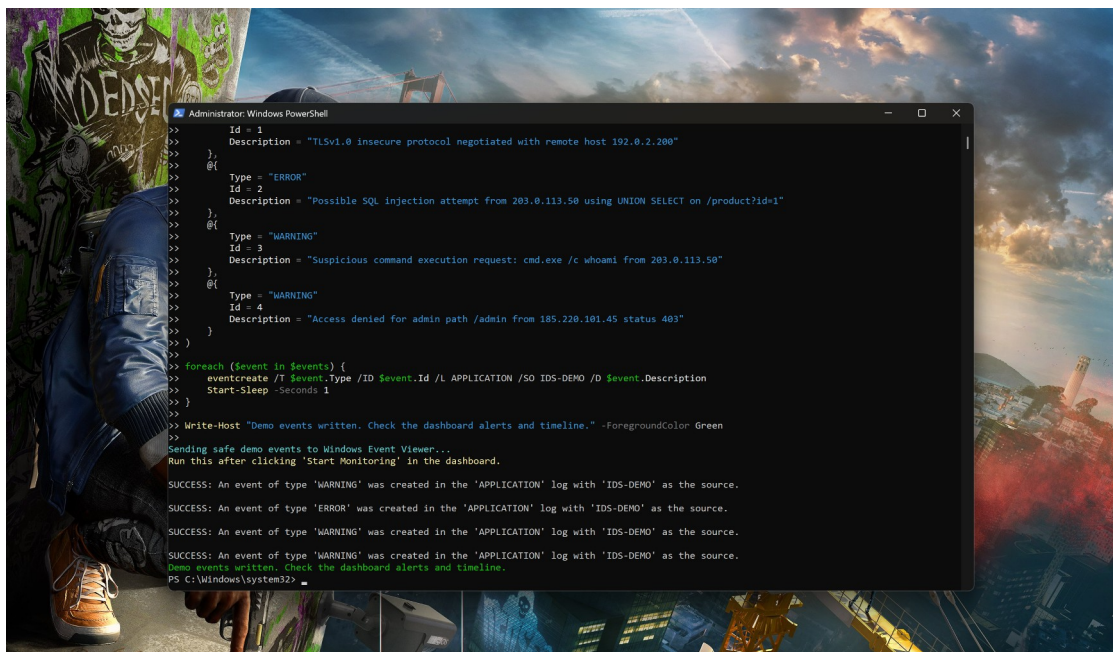
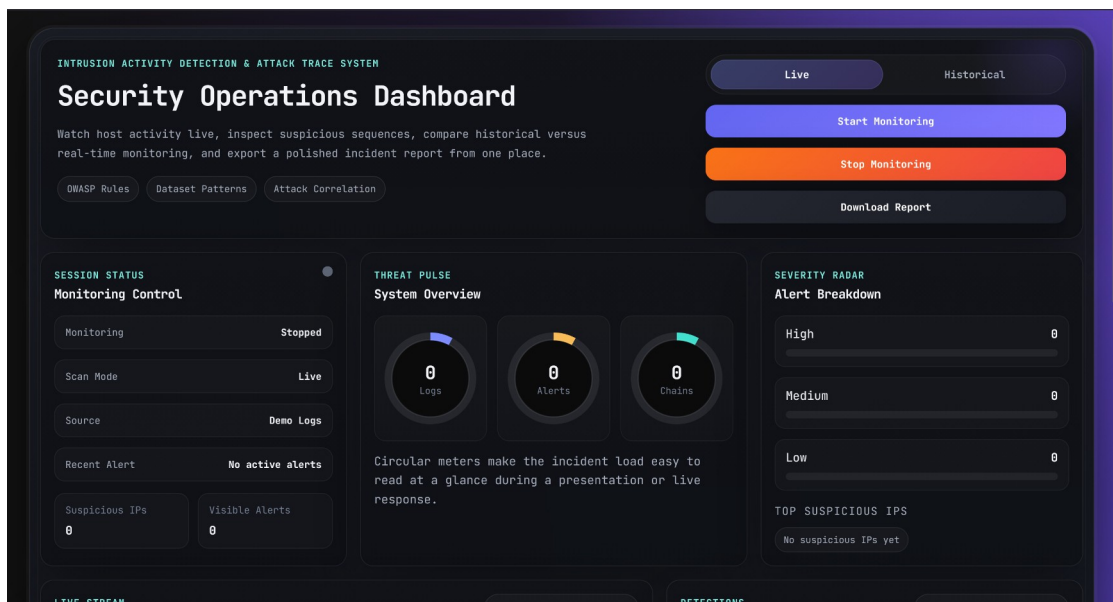
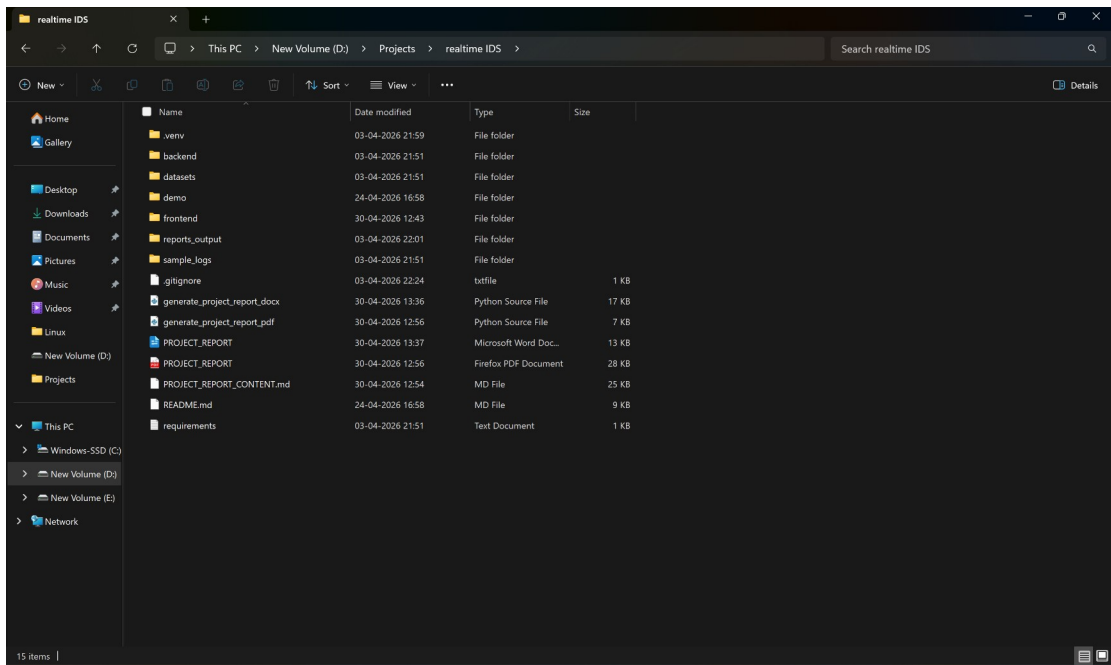
Live mode only processes new activity after monitoring starts. Historical mode also processes existing logs before continuing with real-time updates.

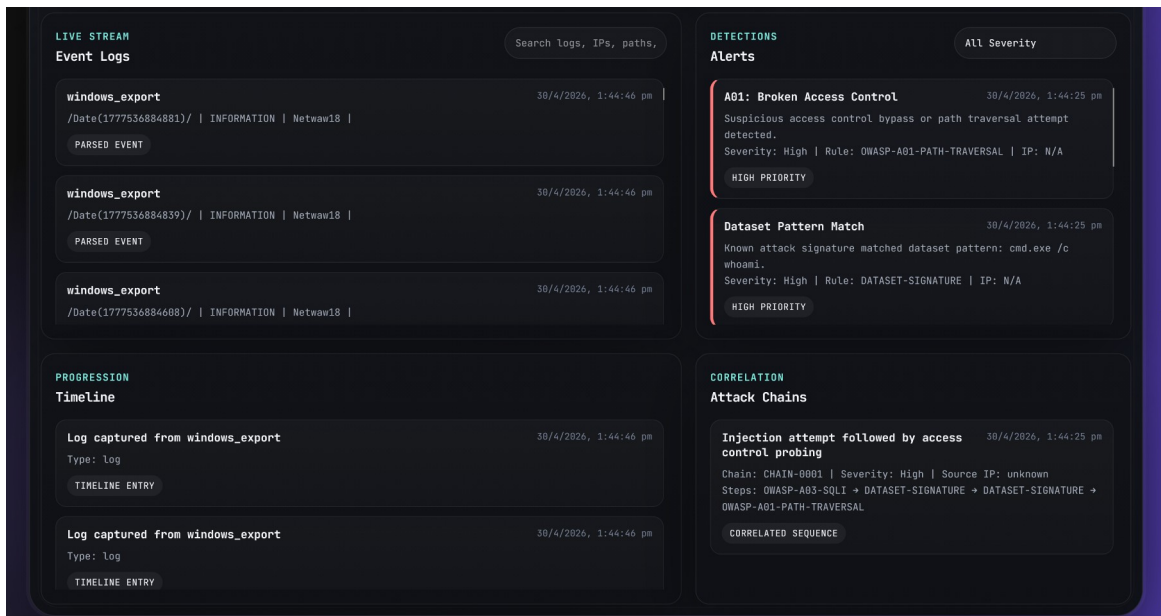
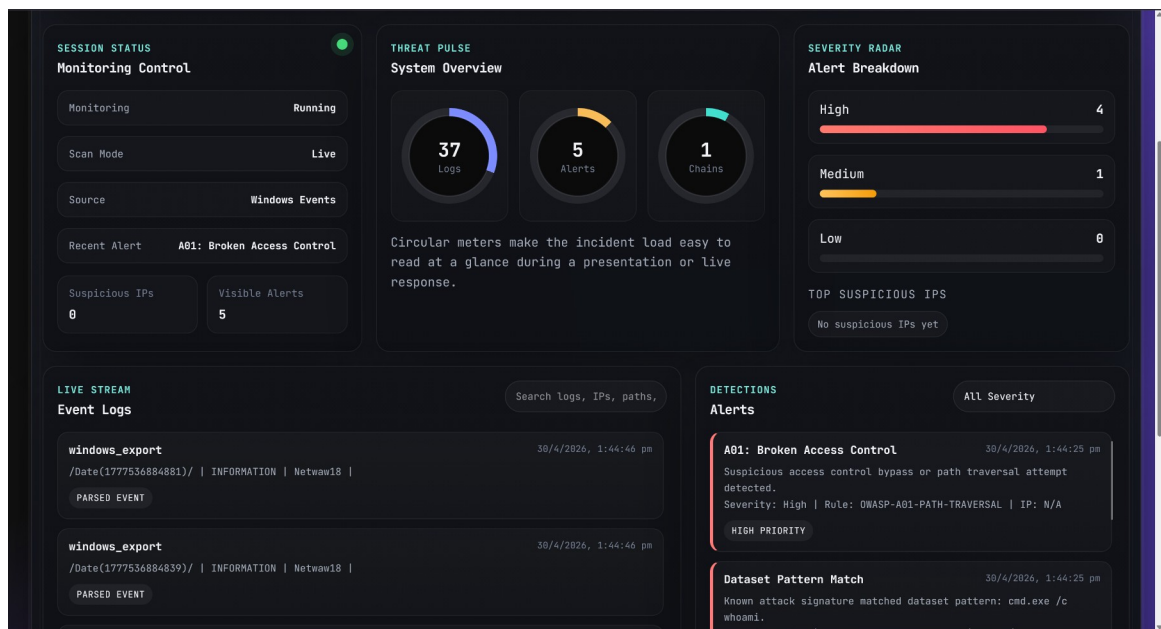
4.4 User Interface

The frontend provides:

- Start Monitoring
- Stop Monitoring
- Live / Historical mode switch
- Logs panel
- Alerts panel
- Timeline panel
- Attack Chains panel
- Download Report button

The dashboard receives updates via WebSocket and periodic API refresh.





4.5 Result Analysis

The developed application successfully demonstrates the complete IDS workflow. During testing, the system was able to:

- capture logs from real or demo sources
- parse logs into structured events
- trigger OWASP-based alerts
- trigger dataset-informed alerts
- correlate related alerts into chains
- show data live in the dashboard

- generate a final PDF report

The system is especially effective for educational and prototype demonstration purposes because it clearly visualizes how raw logs become security alerts and incident traces.

Intrusion Activity Detection Report

A concise incident summary generated from the IDS runtime state, including alerts, correlation results, suspicious IPs, and recent timeline activity.

Generated At	30 Apr 2026, 08:11 AM
Scan Mode	Historical
Source Mode	Windows Events
Monitoring Status	Stopped

Executive Summary

Metric	Value
Total Logs	400
Total Alerts	1
Attack Chains	0
Suspicious IPs	None detected

Severity Breakdown

High	0
Medium	1
Low	0

OWASP Category Summary

Category	Count
A02: Cryptographic Failures	1

Recent Alerts

Time	Severity	Category	Message
30 Apr 2026, 08:11 AM	Medium	A02: Cryptographic Failures	Weak or failed cryptographic control detected in logs.

CHAPTER 5: SOFTWARE TESTING

5.1 Black Box Testing

Black box testing was used to verify the external behavior of the application without focusing on internal code structure.

Black Box Test Cases

Test Case ID	Input / Action	Expected Output	Actual Result	Status
BB1	Click Start Monitoring in Live mode	Monitoring starts and logs begin updating	Monitoring started successfully	Pass
BB2	Click Start Monitoring in Historical mode	Previous logs plus new logs are analyzed	Historical scan started successfully	Pass
BB3	Run Windows or Linux demo script	Suspicious alerts appear in dashboard	Alerts appeared successfully	Pass
BB4	Click Stop Monitoring	Monitoring stops and PDF report is generated	Report generated successfully	Pass
BB5	Click Download Report after stop	PDF report is downloaded	Report downloaded successfully	Pass

5.2 White Box Testing

White box testing was used to verify internal logic paths such as parsing, detection, correlation, and report generation.

White Box Test Cases

Test Case ID	Internal Logic Tested	Expected Behavior	Actual Result	Status
WB1	Parser receives Apache log line	Extract IP, timestamp, path, method, status code	Parsed correctly	Pass
WB2	OWASP detector receives SQLi-style path	Injection alert should be created	Alert created successfully	Pass
WB3	Failed login history reaches brute-force threshold	High severity brute-force alert should be created	Alert created successfully	Pass
WB4	Correlation engine receives repeated auth alerts	Attack chain should be formed	Chain created successfully	Pass
WB5	Report generator is called after stop	PDF should be generated from in-memory state	PDF generated successfully	Pass

CHAPTER 6: CONCLUSION

The ****Intrusion Activity Detection & Attack Trace System**** successfully demonstrates the design and implementation of a software-based mini IDS capable of real-time and historical log analysis. The project combines log parsing, OWASP-inspired rule detection, dataset-informed pattern matching, event correlation, timeline generation, live dashboard visualization, and PDF reporting into a single modular workflow.

The system fulfills the major objectives defined at the beginning of the project. It detects suspicious login behavior, possible injection patterns, cryptographic weaknesses, and suspicious request activity. It also improves interpretability by correlating isolated alerts into attack chains and by presenting chronological evidence to the user through a dashboard and downloadable report.

This project also provides educational value because it demonstrates important cyber security concepts in a practical and understandable way. It shows how a lightweight IDS can be developed without relying on heavy enterprise tools or complex machine learning pipelines.

Future Scope

The following improvements can be added in future:

1. persistent database storage for long-term event history
2. machine learning based anomaly detection
3. email or messaging alerts
4. role-based user authentication
5. richer analytics and charts in the dashboard
6. cloud log integration and distributed monitoring

BIBLIOGRAPHY

1. OWASP Foundation, “OWASP Top 10 Web Application Security Risks,” Available: <https://owasp.org> 2. Canadian Institute for Cybersecurity, “CICIDS Dataset,” Available: <https://www.unb.ca/cic/datasets/> 3. FastAPI Documentation, “FastAPI Framework,” Available: <https://fastapi.tiangolo.com> 4. ReportLab Documentation, “PDF Generation in Python,” Available: <https://www.reportlab.com/documentation/> 5. Microsoft Documentation, “Windows Event Viewer and Event Logs,” Available: <https://learn.microsoft.com> 6. Nginx Documentation, “Access Log Configuration,” Available: <https://nginx.org> 7. Apache HTTP Server Documentation, “Log Files,” Available: <https://httpd.apache.org>

APPENDICES

Appendix A: Software and Tools Used

- Python
- FastAPI
- Uvicorn
- ReportLab
- HTML, CSS, JavaScript
- PowerShell / Linux shell utilities
- CICIDS sample dataset files

Appendix B: Demo Files Included in Project

- `datasets/pattern\_signatures.json`
- `datasets/cicids\_sample.csv`
- `sample\_logs/auth.log`
- `sample\_logs/apache\_access.log`
- `sample\_logs/windows\_security.txt`
- `demo/windows\_event\_demo.ps1`
- `demo/linux\_log\_demo.s`